



Understanding Docker Networking



MetricFire · [Follow](#)

7 min read · Aug 24, 2023

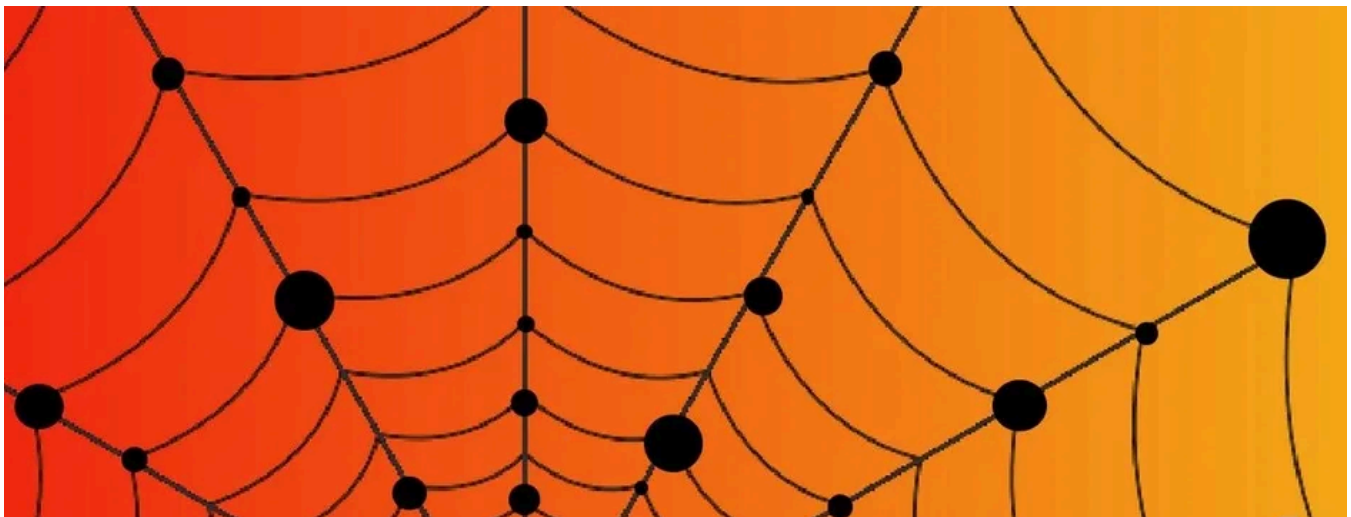


Listen



Share

... More



Introduction

Docker can be used in various use cases: the standalone mode, using Docker Compose, in a single host, or by deploying containers and connecting Docker engines across multiple hosts. The user can use Docker containers with the default network, the host network, or other types of more advanced networks like overlays. This depends on the use case and/or the adopted technologies.

In this tutorial, we are going to understand the different types of container networking. We are also going to study container networking. We will see different types of networking, and finally, we will understand how to extend Docker networking using plugins. This article is the second part of a series. For part, I, check out [this article](#). We also have resources on [Kubernetes Networking](#) and [monitoring Docker containers with cAdvisor](#), for some great further reading.

Key Takeaways

1. Docker offers versatile networking options for different use cases.
2. A default bridge network allows container communication using IP addresses.
3. User-defined bridge networks support automatic service discovery.
4. The host network matches the host's configurations and offers optimized performance.
5. Advanced networking options like Macvlan and overlay networks provide direct connectivity and distributed communication

[Open in app](#) ↗

Medium

 Search



After a fresh Docker installation, you can find a default bridge network up and running.

We can see it by typing *docker network ls*:

NETWORK ID	NAME	DRIVER	SCOPE
5beee851de42	bridge	bridge	local

If you use *ifconfig* command, you will also notice that this network interface is called “docker0”:

```
docker0: flags=4099<UP,BROADCAST,MULTICAST> mtu 1500
    inet 172.17.0.1 netmask 255.255.0.0 broadcast 172.17.255.255
    ether 02:42:90:68:1f:7f txqueuelen 0 (Ethernet)
    RX packets 0 bytes 0 (0.0 B)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 0 bytes 0 (0.0 B)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0
```

All Docker installations have this network. If you run a container, say, Nginx, it will be attached by default to the bridge network:

```
docker run -dit --name nginx nginx:latest
```

You can check the containers running inside a network, by using the “inspect” command:

```
docker network inspect bridge
```

```
---
  "Containers": {
    "dfdbc18945190c832c3e0aaa7013915d77022851e69965c134045bb3a37168c4":
    {
      "Name": "nginx",
      "EndpointID":
    "33a598ffd6d4df792c58a6b6fdc34cd162c9dd3a3f1e58add29f30ad7f1dfdac",
      "MacAddress": "02:42:ac:11:00:02",
      "IPv4Address": "172.17.0.2/16",
      "IPv6Address": ""
    },
  }
```

Docker uses a software-based bridge network that allows containers connected to the same bridge network to communicate while isolating them from other containers not running in the same bridge network.

Let's see how containers running in the same bridge network can connect to each other. Let's create two containers for testing purposes:

```
docker run -dit --name busybox1 busybox
```

```
docker run -dit --name busyboxZ busybox
```

These are the IP addresses of our containers:

```
docker inspect busybox1 | jq -r '[0].NetworkSettings.IPAddress'
docker inspect busybox2 | jq -r '[0].NetworkSettings.IPAddress'
```

```
172.17.0.3
172.17.0.4
```

Let's try to ping a container from another one using one of these IP addresses. For example, ping the container named "busybox1" from "busybox2", using its IP 172.17.0.3.

```
docker exec -it busybox2 ping 172.17.0.3
```

```
PING 172.17.0.3 (172.17.0.3): 56 data bytes
64 bytes from 172.17.0.3: seq=0 ttl=64 time=0.245 ms
```

So, containers on the same bridge can see each other using their IPs. What if I want to use the containers' name instead of the IP.

```
docker exec -it busybox2 ping busybox1
```

```
---  
ping: bad address 'busybox1'
```

We can understand that containers running on the same bridge network are able to see each other using their IP addresses. On the other hand, the default bridge network does not support automatic service discovery.

User-defined Bridge Networks

Using the Docker CLI, it is possible to create other networks. You can create a second bridge network using:

```
docker network create my_bridge --driver bridge
```

Now, attach “busybox1” and “busybox2” to the same network:

```
docker network connect my_bridge busybox1  
docker network connect my_bridge busybox2
```

Retry pinging “busybox1” using its name:

```
docker exec -it busybox2 ping busybox1
```

```
---  
PING busybox1 (172.20.0.2): 56 data bytes
```

```
64 bytes from 172.20.0.2: seq=0 ttl=64 time=0.113 ms
```

We can conclude that only user-defined bridge networks support automatic service discovery. If you need to use service discovery with containers, don't use the default bridge, create a new one.

The Host Network

A container running in the host network matches the networking configurations of the host.

If we take the example of an Nginx image, we can notice that it exposes the port 80.

```
EXPOSE 80
```

If we run the container, we should normally publish the port 80 on another port, say 8080.

```
docker run -dit -p 8080:80 nginx:latest
```

The container is now accessible on port 8080:

```
curl -I 0.0.0.0:8080
```

```
---  
HTTP/1.1 200 OK
```

```
Server: nginx/1.17.5
Date: Wed, 20 Nov 2019 22:30:31 GMT
Content-Type: text/html
Content-Length: 612
Last-Modified: Tue, 22 Oct 2019 14:30:00 GMT
Connection: keep-alive
ETag: "5daf1268-264"
```

When we run the same container on the host network, published ports will be ignored since the port 80 of the host will be used in all cases. If you run:

```
docker run -dit --name nginx_host --network host -p 8080:80 nginx:latest
```

Docker will show a warning:

```
WARNING: Published ports are discarded when using host network mode
```

Let's remove this container and re-run it without publishing its ports:

```
docker rm -f nginx_host;
docker run -dit --name nginx_host --network host nginx:latest
```

You can now run a “curl” on the port 80 of the host machine to check that the web server responds:

```
curl -I 0.0.0.0:80
```

```
HTTP/1.1 200 OK
Server: nginx/1.17.5
Date: Wed, 20 Nov 2019 22:32:27 GMT
Content-Type: text/html
Content-Length: 612
Last-Modified: Tue, 22 Oct 2019 14:30:00 GMT
Connection: keep-alive
ETag: "5daf1268-264"
Accept-Ranges: bytes
```

Since there is no network address translation (NAT), running a container on the host network can optimize performance. When you run a container on the host network, its network will not be isolated from the host, and it will not get its own IP; these are the limits of this type of network.

Macvlan Network

If you are, for instance, developing an application to monitor traffic, which is expected to be directly connected to the underlying physical network, you can use the macvlan network driver. This driver assigns a MAC address to each container's virtual network interface.

Example:

```
docker network create -d macvlan --subnet=150.50.50.50/24--gateway=150.50.50.1
```

When you create a macvlan network, you need to specify the parent, which is the used host interface through which the traffic is physically routed.

None

In some cases, you need to isolate a container even from ingoing/outgoing traffic, you can use this type of network that lacks a network interface.

The only interface the container will have is the local loopback interface (127.0.0.1).

Distributed Networking

Overlay Networks

Your container platform may have different hosts, and in each, there are some containers running. These containers may need to communicate with each other. This is when overlay networks are useful.

The overlay network is a distributed network created among multiple Docker daemons in different hosts. All containers connected to this network can communicate.

Ingress

Docker Swarm, for example, uses overlay networking to handle the traffic between swarm services.

In order to test this, let's create 3 Docker Machines (a manager + 2 workers):

```
docker-machine create manager  
docker-machine create machine1  
docker-machine create machine2
```

After configuring a different shell for each of these machines (using `eval $(docker-machine env <machine_name>)`), you initialize Swarm on the manager, using the following command:

```
docker swarm init --advertise-addr <IP_address>
```

Don't forget to execute the join command on both workers:

```
docker swarm join --token xxxx <IP_address>:2377
```

If you list the networks on each host using `docker network ls`, you will notice the existence of an overlay ingress network:

```
o5dnttidp8yq    ingress    overlay    swarm
```

On the manager, create a new service with three replicas:

```
docker service create --name nginx --replicas 3 --publish published=8080,target=8080
```

To see the swarm nodes where your service has been deployed, use `docker service ps nginx`.

ID	NAME	IMAGE	NODE
fmvfo2nschcq	nginx.1	nginx:latest	machine1
pz1kob42tqoe	nginx.2	nginx:latest	manager
xhhnq68sm65g	nginx.3	nginx:latest	machine2

In my case, I have a container running in each of my nodes.

If you inspect the ingress networking using `docker inspect ingress`, you will be able to see the list of peers (swarm nodes) connected to the same network:

```
"Peers": [
  {
    "Name": "3a3c4007c923",
    "IP": "192.168.99.102"
  },
  {
    "Name": "9827ad03b358",
    "IP": "192.168.99.100"
  },
  {
    "Name": "60ae1df1c8b2",
    "IP": "192.168.99.101"
  }
]
```

The ingress network is a particular type of overlay network created by default.

When we create a service without connecting it to a user-defined overlay network, it connects by default to this ingress network. Using Nginx, here is an example of a service running without a user-defined overlay network but connected by default to the ingress network.

```
docker service inspect nginx|jq -r .[0].Endpoint.Ports[0]
```

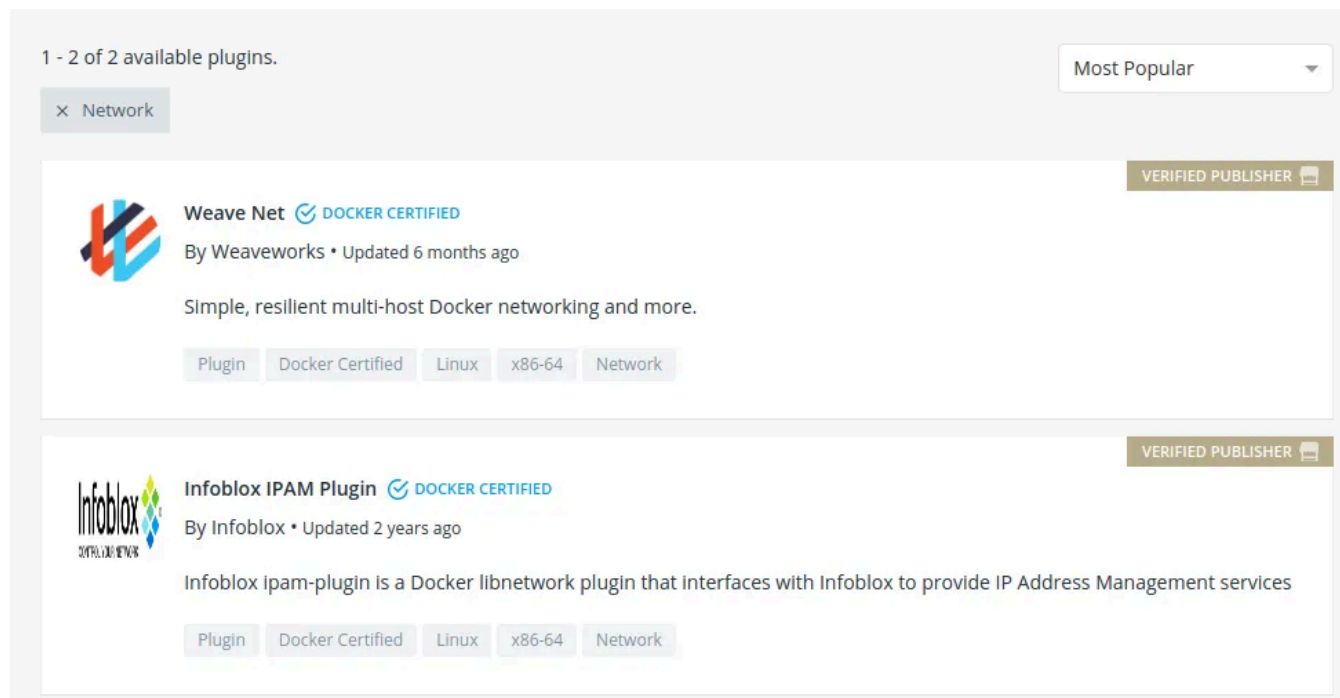
```
---
{
  "Protocol": "tcp",
  "TargetPort": 80,
  "PublishedPort": 8080,
  "PublishMode": "ingress"
}
```

Docker Network Plugins

There are other types of use cases that need new types of networks. You may also want to use another technology to manage your overlay networks and VXLAN tunnels.

Docker networking is extensible and allows using plugins to extend the features provided by default.

You can use or develop networking plugins. You can also find some verified networking plugins on Docker Hub.



Each plugin has different use cases and installation procedures. For example, Weave Net is a virtual Docker network that connects containers across a cluster of hosts and enables automatic discovery.

To install it, you can follow the official instructions on the Docker Hub:

```
docker plugin install --grant-all-permissions store/weaveworks/net-plugin:2.5.2
```

You can create a Weave Net network and make it attachable using:

```
docker network create --driver=store/weaveworks/net-plugin:2.5.2 --attachable
```

Then, create containers using the same network:

```
docker run -dit --rm --network=my_custom_network -p 8080:80 nginx
```

Part III of this series is all about Kubernetes Networking, [check it out here](#). If you missed part I, [check it out here](#). If you're interested in monitoring your Docker containers, learn how to do it in our article about [monitoring Docker with cAdvisor](#). Also, check out our blog post on [Monitoring Kubernetes with Prometheus](#).

[Docker](#)[Networking](#)[Monitoring](#)[Observability](#)[Data Visualization](#)[Follow](#)

Written by MetricFire

156 Followers

Time series monitoring as a service using Graphite and visualized on Grafana.

More from MetricFire